PRACTICAL 1

Title - Setting up a company that sells testing services to software houses

Problem Statement -

① What is the testing process that will be followed in the company?
② What is the focus of testing services?
③ What kind of people are you going to hire as staff for the company?
④ How are you going to validate that testing project carried out in the company has been beneficial to the customer?
⑤ What kind of automated tools will company use?

Solution -

→ ① The testing process that will be followed by company are:
a. Prepare for testing a s/w system
b. Plan tests that will be conducted on s/w system
c. Execute steps as defined in the test plan
d. Conduct acceptance testing by s/w system users
e. Analyse that results and report them to appropriate s/w system stakeholders
f. Test installation and test changes made to s/w after it is placed into operational environment
g. Conduct post-implementation analysis to evaluate the effectiveness and efficiency of test process

→ ② The focus of testing services will be on -

a. Software testing should reduce s/w development risk. Risk is present in all s/w development projects and testing reduces those risks

b. Testing should be performed effectively. Testing should be performed in a manner in which the maximum benefits are achieved.

c. Testing should unover defects. At the conclusion there should be no defects in the s/w

d. Testing should be performed using business logic. Money should not be spent on testing unless it can be spent economically to reduce business risk

e. Testing should occur throughout the development life cycle. Testing is not a phase but a process. It begins when development begins and ends when software is no longer being used

f. Testing should test both structure and function. Should test functional requirements to ensure they are correct and test the software structure to process those functional requirements in an effective and efficient manner

SANSKAR SARIAN
EDUCATION SOCIETY

→ ③ The company will recruit people with foll qualities-

a. A clear communicator: A defect report is no good if we can't understand it
b. Patient : Sometimes it takes a lot of back-and-forth to get to the root of a problem. And programmers have ego's they'll often try to push issues back to the testers
c. Passionate : The best developers are the one that really care about development and maybe even get little excited about it.
d. Creative : Tries unusual, non-standard ways to use the software. Instead of just following scripted steps, a creative tester thinks outside the box to find hidden bugs that normal users might cause
e. Analytical : Not only spots bugs but also figures out exact, simple steps to reproduce them. When a bug is intermittent, the analyst writes a clear sequence so developers can see and fix it easily
f. Not a programmer : Focuses on real, hands-on testing of the product. The job is to make sure the software works correctly, not to spend time writing automated test scripts like programmer would

SANSKAR SARTAN
EDUCATION SOCIETY

	Yes	No	N/A	Comments
1) Does management support concept of continuous improvement to test processes?	✓			Management support improvement
2) Have resources been allocated to improve test processes?	✓			Resource provided
3) Has single individual been appointed responsible for overseeing the improvement of test processes?		✓		No dedicated person
4) Have the results of testing been accumulated over times?	✓			Test results stored
5) Do results of testing include types of items identified in input section of this chapter?	✓			Required items included
6) Do testers have adequate tools to summarize, analyze and report the results of previous testing?		✓		Tools not sufficient
7) Do the result of analysis appear reasonable?	✓			Analysis reasonable
8) Is the analysis performed on daily basis?		✓		Not done regularly
9) Are the results of the analysis incorporated into proved test process?	✓			Improvements applied
10) Is data maintained so there can be determination as to whether those installed improvements do in fact improve test processes?		✓		Improvement tracking not maintained

→ ⑤ Write about any 5 automation tools -

a. Selenium
b. AutoIT
c. Bugzilla Software Testing
d. QTP
e. WAPT
f. Win runner
g. VTest

SANSKAR SARJAN
EDUCATION SOCIETY

PRACTICAL 2

Title - Write SRS of Hotel Management System

Problem Statement -
Prepare a software requirement specification of 'Hotel Management System'

Solution -

Software Requirement Specification

Deficiency	Comments & Suggestions
There is no provision for changed rate & customers feedback	There should be module to show changed rate and space for customer's feedback
There was a confusion in the teams that who is customer and who is employees of the hotel	We can use code name system for employees of the hotel to avoid confusion

Function Requirement

1. Reservation / Booking

1.1 The system shall record reservations
1.2 The system shall record customer's first name
1.3 The system shall record customer's last name
1.4 The system shall record no. of occupants
1.5 The system shall record room number
1.6 The system shall display default room rate
1.6.1 The system shall allow default room rate to be changed
1.6.2 The system shall require a comment to be entered, giving the reason for changing default room rate
1.7 The system shall record customer's phone number
1.8 The system shall display whether or not the room is

guaranteed

1.9 The system shall generate unique confirmation number for each reservation

1.10 The system shall automatically cancel non-guaranteed reservations if the customer has not provided their credit card number by 6:00 on check-in time, date

1.11 The system shall record the expected check-in date and time

1.12 The system shall record the expected check-out date and time

1.13 The system shall check-in customers

1.14 The system shall allow reservations to be modified without having to re-enter all customer information

1.15 The system shall check-out customers

1.15.1 The system shall display the amount owed by customer

1.15.2 To retrieve customer information that last name or room number shall be used

1.15.3 The system shall record that room is empty

1.15.4 The system shall record the payment

1.15.5 The system shall record payment type

1.16 The system shall charge customer for an extra night if they checkout after 11:00 am

1.17 The system shall mark guaranteed rooms as "must pay" after 6:00pm on check-in date

1.18 The system shall record customer feedback

2. <u>Food</u>

2.1 The system shall track all meals purchased in hotel

2.2 The system shall record payment and payment type for meals

2.3 The system shall bill current room if payment is not made at time of service

2.4 The system shall accept reservations for restaurant and room service

3. Management

3.1 The system shall display the hotel occupancy for a specified period of time
3.2 The system shall display projected occupancy for period of time
3.3 The system shall display room revenue for a specified project/period of time
3.4 The system shall display food revenue for a specified period of time
3.5 The system shall display an exception report, showing where default room and food prices have been overridden
3.6 The system shall allow for addition of info regarding rooms, rates, menu items, prices & user profiles
3.7 The system shall allow for deletion of info regarding rooms, rates, menu items, prices & user profiles
3.8 The system shall allow for modification of info regarding rooms, rates, menu items, prices & user profiles
3.9 The system shall allow managers to assign user Password

PRACTICAL 3

Title - Black-Box Testing, Equivalence Patitioning and Boundary Value Analysis

Problem Statement -
The program reads an arbitrary no. of temperatures within range -60°C ..... +60°C and prints their mean value. Design test cases for testing program with black-box strategy.

Theory -
The first of dynamic testing techniques are specification based testing techniques. These are also known as 'black-box' because they view s/w as black-box with inputs and outputs, but they have no knowledge of how the system or component is structured inside the box.
Functional testing is concerned with what the system does, its features or functions. Non-functional testing is concerned with examing how well the system does something rather than what it does. Non-functional aspects includes performance, usability, portability, maintainability etc. Techniques to test these non-functional aspects are less procedural and less formulized than those of other categories as actual tests are more dependant on type of system, what it does and resource available for tests
The four specification-based techniques are:

• Equivalence Partitioning (EP)

It is a good all-round specification-based black-box technique. It can be applied at any level of testing & is often a good technique to use first.
EP is a testing technique where you divide test conditions into groups that are treated the same by the system. These groups are called equivalence classes or partitions EP technique then requires that we need test only one condition from each partition. This is because we are

assuming the same way by the software. If one condition works we assume all of the conditions will work. If one of the conditions does not work then we assume that none of the conditions will work so again there is little point in testing any more in that partitions.

- Boundary Value Analysis -

It is used on testing at boundaries between partitions. If you have even done 'range checking' you were probably using the boundary value analysis technique even if you were not aware of it.

- Decision tables

- State Transition Testing

Solution -

The program accepts integers b/n -60 and +60 so we apply Equivalance Partitions and Boundary Value Analysis

Equivalence Partitions :

< -60	-60 to +60	> 60
Invalid Partition	Valid Partition	Invalid Partition

Boundary Value Analysis :

-61	-60	-59
Invalid	Valid	Valid
59	60	61
Valid	Valid	Valid

Test Cases:

Test Case ID	Input Value	Remark
1	-61	Invalid Boundary
2	-100	Invalid Partition
3	-60	Valid Boundary
4	0	Valid Partition
5	+61	Invalid Boundary
6	150	Invalid Partition

Conclusion:

As the program specification are given. With given specification the above test cases are generated and seem to work well

SANSKAR SARJAN
EDUCATION SOCIETY

PRACTICAL 4

Title - Black-Box Testing, Equivalence Partitioning and Boundary Value Analysis

Problem Statements -
When getting a person's weight and height as input the program prints person's body weight index. The weight is given in kilometers (as real number, for instance: 82.0) and height in meters (as real number for instance: 1.86). The body weight index equals weight divided by height squared : weight / (height * height). Design test cases for testing the person with black-box strategy.

Program -

```
#include <stdio.h>
#include <conio.h>
void main ()
{
   float ht, wt, bmi;
   clrscr();
   ht=0; wt=0;
   while (ht < 0.3 || ht > 2.4)
   {
      printf("Enter valid height in meters");
      scanf(" %f", &ht);
   }
   while (wt < 2.0 || wt > 350)
   {
      printf("\n Enter valid weight in kilograms");
      scanf("%f", &wt);
   }
   bmi = wt / (ht * ht);
   printf("\n BMI = %f kg/sqm", bmi);
   getch();
}
```

Equivalence Partition (ht):

<0.3	0.3 to 2.4	>2.4
Invalid Partition	Valid Partition	Invalid Partition

Equivalence Partition (wt):

<2.0	2.0 to 350.0	>350.0
Invalid Partition	Valid Partition	Invalid Partition

Boundary Value Analysis (ht):

0.29	0.3	0.31
Invalid	Valid	Valid
2.39	2.40	2.41
Valid	Valid	InValid

Boundary Value Analysis (wt):

1.99	2.0	2.01
Invalid	Valid	Valid
349.99	350	350.01
Valid	Valid	Invalid

Test Cases :

Test Case ID	ht(input)	wt(input)	Remark	Expected output
1	0.29		Invalid boundary/partition	Enter valid ht
2		1.99	Invalid boundary/partition	Enter valid wt
3	0.30	2.00	Valid boundary/partition	22.22kg/sq.m
4	2.39	349.99	Valid boundary/partition	61.27kg/sq.m
5	2.40	350	Valid boundary/partition	60.76kg/sq.m
6	2.41		Invalid boundary/partition	Enter valid ht
7		350.01	Invalid boundary/partition	Enter valid wt
8	1.86	86.00	Invalid boundary/partition	24.85 kg/sq.m

Conclusion :

The program was tested with above test cases and worked well

SANSKAR SARJAN
EDUCATION SOCIETY

PRACTICAL 5

Title : Black Box Testing : Decision table and Cause Effect Graphing

Problem Statement:
An insurance agency has the following norms fixed to provide premium for its policy holders

1. If age <= 25 and no claim has been made, premium increase will be $50, else $25
2. If age <= 25 and no.of claims made is one, premium increase will be $100, else $50
3. If age <= 25 and no.of claims made is 2-4, premium increase will be $400, else $200
4. If one or more claims are made, send warning letter. If the no.of claims made is 5 or more, cancel policy

Draw decision table and cause effect graph for insurance renewal

Theory:
The techniques of equivalence partitioning and boundary values analysis are often applied to specific situations or inputs.
A decision table is a good way to deal with combinations of things. This technique is sometimes also referred to as a 'cause-effect' table. The reason for this is that there is an associated logic diagramming technique called 'cause-effect graphing' which was sometimes used to help derive decision table.
Decision tables aid the systematic selection of effective test cases and can have beneficial side-effect of finding problems and ambiguities in the specification. It is a software testing technique that works well in conjunction with equivalence partitioning.

According to Myer Cause & Effect Graphing is done through the foll steps:

step 1: For a module, identify input conditions and actions

step 2: Develop a cause-effect graph

step 3: Transform cause-effect graph into a decision table

step 4: Convert decision table rules to test cases. Each column of decision table represents test case

PRACTICAL 6

Title : State Transition Testing

Problem Statement:
Specifications
The s/w responds to input requests to change display mode for time display device
The display mode can be set to one of the four values
Two corresponding to displaying either time or date
The other two when altering either time or date
Four possible input requests:
Change mode (CM)
Reset (R)
Time Set (TS)
Date Set (DS)

Change Mode (CM):
Activation of this shall cause display mode to move between "Display Time (T)" and "Display Date (D)"

Reset (R):
If display mode is set to T or D, then a "reset" shall cause the display mode to be set to "Alter time (AT)" or "Alter Date (AD)" modes

Time Set (TS):
Activation of this shall cause display mode to return to T from AT

Date Set (DS):
Activation of this shall cause display mode to return to D from AD

Draw state transition diagram, possible transitions table, state table and write test cases

Theory :

State Transition testing is used where some aspect of the system can be described in what is called 'finite state machine'. This means that system can be in no.of different states and transitions from one state to another are determined by rules of 'machine'. Any system where you get a different output for same input, depending on what has happened before, is a finite state system.

A state transition model has four basic parts:
- the states that slw may occupy;
- the transitions from one state to another;
- the events that cause transition;
- the actions that result from a transition

Conclusion:

The state transition table is drawn and test cases are developed

SANSKAR SARJAN EDUCATION SOCIETY

PRACTICAL 7

Title : Branch - Decision - Condition Coverage

Problem Statement :
For foll liability procedure, design test cases using branch, condition, decision and multiple decision coverage

```
Procedure Liability (Age, Gender, Married, Premium)
Begin
    Premium := 500;
    If (Age < 25) and (Gender = Male) and (not Married)
        Then Premium = Premium + 1500;
    Else (if (Married or Gender = Female))
        Then Premium = Premium - 200;
    If (Age > 45) and (Age < 65)
        Then Premium = Premium - 100;
End;
```

Theory :
Statement Coverage -
Make every line of code run at least once. It checks if the code executes, but it's not enough to find all bugs.

Branch / Decision Coverage -
Test each decision (like if, else) for both true and false paths. Every branch must be taken once

Condition / Decision Coverage -
Test each Boolean part inside a decision separately. Test every condition for true and false, ignoring internal logical links

Multiple Condition Coverage -
Check every possible mix of all Boolean sub-conditions. You need many test cases to cover all combinations

Conclusion:
All types of test cases are designed for testing

SANSKAR SARTAN
EDUCATION SOCIETY